

LLM-Enhanced, Multimodal Welding Process Optimization

Project	LLM-Enhanced, Multimodal Welding Process Optimization with Explainable AI
Domain	MIG / MAG / TIG / SMAW arc welding — smart manufacturing, Industry 4.0
Type	Operator-facing diagnostic & advisory chatbot (MVP, implemented & validated)
Author	Tasnimur Rahman Fayad (ID: 2009026)
Supervisor	Kazi Naimur Rahman — Assistant Professor, Dept. of MIE, CUET
Status	MVP implemented and validated — 88/88 unit tests passing
Version	1.0 · June 2026

This document describes the project for academic review: what it is, the problem it solves, what was implemented, the technologies used, and the validation results.

Table of Contents

1. Executive Summary
2. Problem Statement & Motivation
3. Solution Overview
4. System Architecture
5. The Four Answer Routes
6. Machine Learning & Anomaly Detection
7. Explainable AI (XAI) Layer
8. Retrieval-Augmented Generation (RAG) Knowledge Layer
9. LLM Synthesis Layer
10. Technology Stack
11. Dataset
12. Evaluation & Results
13. User Interfaces
14. How to Run
15. Limitations & Future Work
16. Conclusion

1. Executive Summary

This project is an **operator-facing intelligent chatbot** for arc-welding shops that answers natural-language questions about welding operations. It unifies three heterogeneous data sources common to modern welding cells — **time-series sensor feeds**, structured **event/production logs**, and free-text **operator notes** — and combines classical machine learning, welding physics, explainable-AI techniques, and a Large Language Model (LLM) into a single conversational assistant.

The central design principle is **trustworthy, hallucination-resistant AI**. In a safety-critical welding context, fabricated numbers are unacceptable. Therefore every numeric output is computed by **deterministic physics or a trained ML model — never by the LLM**. Knowledge answers are grounded in a **retrieval-augmented (RAG) corpus with inline source citations**. The LLM is used *only* to phrase grounded results in clear, plain language. Every recommendation is accompanied by an explanation (SHAP feature attribution, parameter sensitivity tables, confidence scores, or cited sources).

The system was validated on a controlled synthetic, tri-modal dataset. It achieves an anomaly detection **F1 of 0.95** and **ROC-AUC of 0.999** under 5-fold cross-validation, **100% in-window compliance** for recommended weld parameters, **zero fabricated numbers**, and passes **88 of 88** unit tests.

2. Problem Statement & Motivation

Modern welding shops generate large volumes of heterogeneous data, yet operators still struggle to:

- choose **optimal weld parameters** (current, voltage, travel speed, wire feed) for a given job;
- **diagnose faults** quickly when a station behaves abnormally;
- obtain **trustworthy, explainable** guidance they can act on with confidence.

Conventional machine-learning tools behave as opaque **black boxes** that output a prediction without justification, while raw LLMs can **hallucinate unsafe numbers**. Neither is acceptable on a shop floor where an incorrect heat-input figure can ruin a weld or create a safety hazard. This project addresses that gap by separating **computation** (physics + ML, fully auditable) from **communication** (the LLM, phrasing only), and by attaching an explanation to every answer.

3. Solution Overview

The chatbot answers welding and manufacturing questions through a set of specialised, grounded routes. An **intent guard** first rejects out-of-scope questions (general knowledge, jokes, geography, etc.). In-scope questions are then dispatched by a **router** to the appropriate engine:

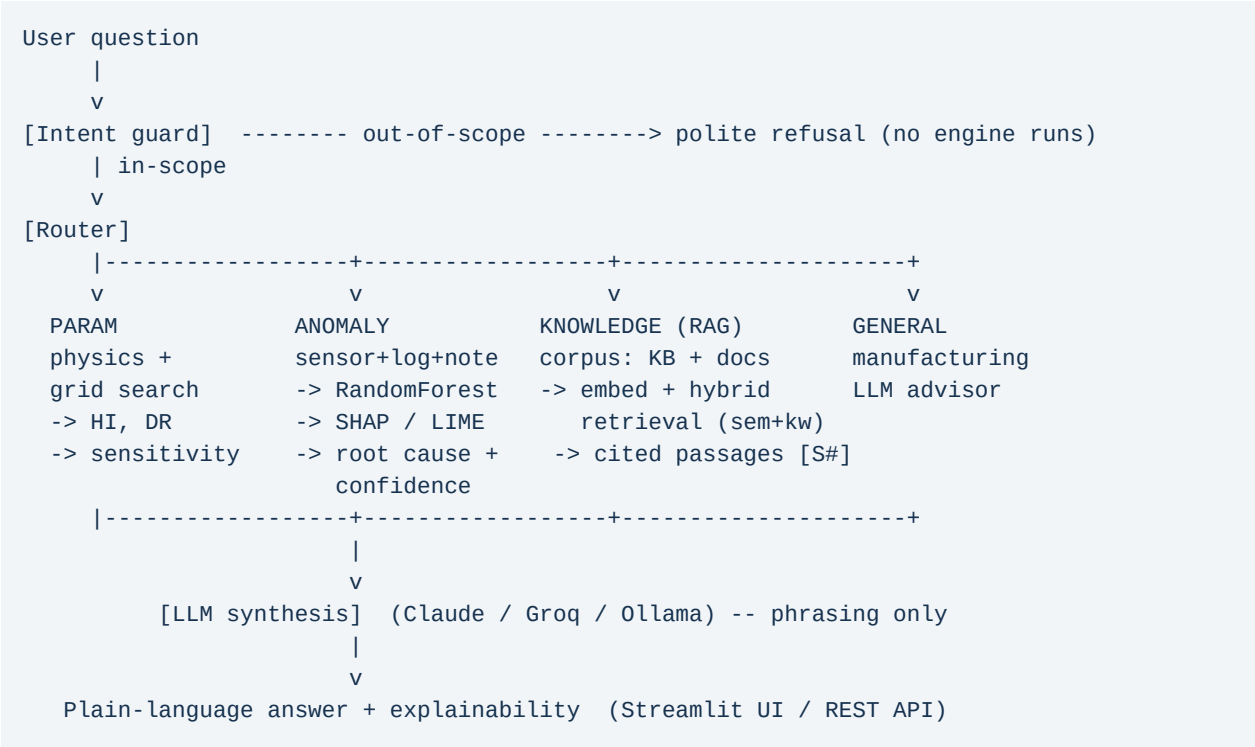
Capability	How it is grounded	Explainability
Parameter optimization	Welding physics + deterministic grid search	Sensitivity table
Fault diagnosis	Supervised ML on fused sensor/log/note data	SHAP + LIME
Welding knowledge	Hybrid RAG retrieval over a welding corpus	Inline [S#] citations
General manufacturing	LLM advisory (numbers flagged to verify)	Stated assumptions

The result is an assistant that is **transparent, traceable, grounded, and hallucination-resistant**: numeric outputs come from physics/ML, knowledge answers from cited retrieval, and the LLM never

invents figures.

4. System Architecture

The system follows a clean pipeline: **intent guard** → **router** → **route-specific engine** → **LLM synthesis** → **answer**. Numeric routes are fully deterministic and auditable; only the final phrasing passes through the LLM.



4.1 Key Modules

Module	Location	Responsibility
Intent classifier	src/chat/intent.py	Scope gate (keyword + regex, no LLM call)
Router	src/api/pipeline.py	route_question() — picks the engine
Parameter advisor	src/reasoning/param_advisor.py	Physics + grid-search optimization
Anomaly detector	src/model/anomaly.py	RandomForest classifier
DistilBERT detector	src/model/bert_detector.py	Fine-tuned transformer (optional 2nd detector)
SHAP explainer	src/explain/shap_explainer.py	Feature attribution
LIME / attention	src/explain/	Independent XAI cross-checks
RAG retriever	src/rag/ (corpus, index, retriever)	Corpus build, embeddings, hybrid retrieval
LLM synthesis	src/chat/synthesize.py, prompts.py	Narration, pluggable backends
Frontend	app/streamlit_app.py	Chat UI (dark industrial theme)
API	src/api/main.py	REST endpoints (FastAPI)

5. The Four Answer Routes

5.1 Parameter Optimization (primary deliverable)

Given a material, thickness and process, the advisor looks up the published standards window (AWS D1.1 / ISO 15614-1 style), then runs a **deterministic grid search** to find the best-efficiency settings. Two welding-physics quantities drive the objective:

```
Heat input:      HI = (eta * 60 * I * V) / (1000 * S)      [kJ/mm]
Deposition rate: DR = WFS * A * rho * eta_d               [g/min]

Objective (maximised): deposition_rate x heat_input_window_score
where I = current, V = voltage, S = travel speed, WFS = wire feed speed.
```

It returns the optimal current/voltage/speed/wire-feed/gas-flow, the resulting heat input and deposition rate, an efficiency score, and a **sensitivity table** (each parameter is nudged ± 1 step and the qualitative effect listed). For unsupported materials (e.g. titanium, copper) or invalid combinations (e.g. SMAW on aluminium) it returns an **honest error rather than fabricated numbers**.

Example — 5 mm stainless steel, MIG: 187 A, 24.2 V, 450 mm/min, 9.3 m/min wire feed; heat input 0.493 kJ/mm (window 0.30–0.65); deposition 78.5 g/min; efficiency 8/10.

5.2 Fault Diagnosis

When a question references a specific station/line/time, the system loads and fuses the sensor, log and operator-note data for that station, detects whether the window is anomalous, classifies the fault, explains it with SHAP, maps it to a likely root cause and recommended action, and attaches a confidence score. Five fault types are supported: **arc instability**, **wire-feed fault**, **gas-flow failure**, **overheating**, and **underheat**.

5.3 Welding Knowledge (RAG)

Defect, troubleshooting, quality, productivity and cost questions are answered by a **hybrid retrieval-augmented generation** layer: the most relevant passages are retrieved from a welding corpus (curated knowledge base + drop-in reference documents) using combined semantic-embedding and keyword search, and the LLM answer is grounded strictly in those passages with inline **[S#] source citations**. (Detailed in Section 8.)

5.4 General Manufacturing & Simulation Integration

Questions about non-welding manufacturing domains (CNC machining, injection moulding, casting, forming, additive manufacturing, quality/operations) are routed to a more capable LLM advisor (Claude Opus). Because there is no deterministic grounding here, the prompt instructs the model to flag specific numbers as **starting points to verify**. A further specialised persona answers **robotics / simulation-integration** questions (NVIDIA Isaac Sim, digital twins, sensors, teleoperation, data acquisition) relevant to the project's future direction.

6. Machine Learning & Anomaly Detection

The diagnosis engine runs a **load** → **preprocess** → **fuse** → **detect** → **explain** → **reason** → **confidence** pipeline. Per-modality preprocessing imputes and denoises sensor signals into 30-minute windows, counts log event types and alarm flags, and embeds operator notes. The modalities are then **temporally aligned**

and fused (pandas merge_asof) into a single feature matrix.

The primary detector is a **RandomForest classifier** (scikit-learn) — chosen because it is accurate on tabular fused features and natively compatible with the SHAP TreeExplainer. An **optional fine-tuned DistilBERT** transformer detector is also implemented: each fused window is serialised to a short text description and classified by a fine-tuned sequence model, providing a like-for-like comparison and demonstrating the LLM fine-tuning path. A **confidence score** combines model certainty and explanation agreement:

```
confidence = 0.7 x anomaly_probability + 0.3 x SHAP_agreement
```

7. Explainable AI (XAI) Layer

Explainability is a first-class requirement — no prediction is presented without justification. Three complementary techniques are used:

- **SHAP (TreeExplainer)** — primary attribution; surfaces the top sensor channels that drove each anomaly prediction, with a waterfall plot in the UI.
- **LIME** — an independent local-surrogate cross-check, run as a second opinion on the same window.
- **Attention visualisation** — highlights the salient tokens in the relevant operator note.

A heuristic **root-cause mapper** then matches the SHAP driver pattern (plus log event codes) to a ranked list of likely causes, and a **recommendation engine** maps each fault type to a primary corrective action and urgency level (config/cause_action_map.yaml).

8. Retrieval-Augmented Generation (RAG) Knowledge Layer

The welding-knowledge route upgrades brittle keyword matching to **semantic retrieval with grounded, cited generation**. The pipeline has three stages:

- **Corpus** (src/rag/corpus.py) — one passage per topic from the curated knowledge YAML, plus reference documents in data/knowledge_docs/ chunked by section.
- **Index** (src/rag/index.py) — a local sentence-transformers encoder (all-MiniLM-L6-v2) produces L2-normalised embeddings persisted in models/rag_index/; cosine search via dot product. The index rebuilds automatically only when the corpus content hash changes.
- **Retriever** (src/rag/retriever.py) — hybrid ranking blends semantic and lexical signals and tags each returned passage for citation:

```
score = 0.7 x semantic(cosine) + 0.3 x lexical(word overlap)
-> top-k passages, each tagged [S1], [S2], ... for inline citation
```

Key design choices: **local embeddings** (free, fast, offline-capable — no data leaves the machine, no cloud vector DB); **hybrid retrieval** (semantic catches paraphrases, lexical catches exact defect/parameter names); **inline [S#] citations** for full traceability; and a **keyword fallback** so the route degrades gracefully if the embedding model cannot load.

9. LLM Synthesis Layer

The LLM converts the structured engine output into a plain-language answer — and nothing more. A strict system prompt forbids it from producing any number not present in the computed payload. If no LLM backend is configured, a **deterministic text fallback** still returns the grounded result (computed values or cited RAG passages). The layer is backend-pluggable:

Backend	Model (default)	Role
Anthropic Claude	claude-haiku-4-5 (narration)	Default — fast, low-cost phrasing
Anthropic Claude	claude-opus-4-8 (general route)	Higher-capability advisory answers
Groq	llama-3.3-70b-versatile	Fast hosted alternative
Ollama	llama3.2 (local)	Fully offline operation

The backend is selected via the SYNTHESIZER_BACKEND environment variable, and falls back through the chain if the chosen backend has no credentials, so a demo always runs.

10. Technology Stack

Area	Technologies
Language / runtime	Python 3.11 (pure-Python, Windows-friendly, no GPU required)
Data	NumPy, pandas, PyArrow (Parquet)
Machine learning	scikit-learn (RandomForest), joblib
NLP / transformers	sentence-transformers (all-MiniLM-L6-v2), Hugging Face Transformers, Accelerate (DistilBERT fine-tuning)
Explainability (XAI)	SHAP, LIME, Matplotlib
LLM backends	Anthropic (Claude), Groq (Llama 3.3), Ollama (local), python-dotenv
Validation / config	Pydantic, PyYAML
Backend API	FastAPI, Uvicorn
Frontend	Streamlit (dark industrial theme)
Testing	pytest, pytest-cov (88 tests)

11. Dataset

A fully **synthetic, tri-modal dataset** is generated programmatically with NumPy and pandas. Synthetic generation was a deliberate decision: real factory data is sensitive and rarely tri-modal, whereas synthetic generation gives full control over modalities, anomaly types and **exact ground-truth labels**, making evaluation meaningful. The pipeline is data-agnostic — real CSVs can be swapped in directly.

Modality / file	Contents	Size
sensors.csv	5 channels: current, voltage, speed, wire-feed, gas-flow	30,240 rows
logs.csv	Structured event / maintenance / alarm records	278 rows

Modality / file	Contents	Size
notes.csv	Templated free-text operator shift notes	31 rows
ground_truth.csv	Per-window labels: anomaly type + root cause	30 windows
fused.parquet	Aligned & fused 30-min feature windows	2,010 windows (62 anomalous)

Scope: 3 welding stations × 7 days at 1-minute resolution; materials = mild steel / stainless steel / aluminium; processes = MIG/MAG (GMAW), TIG (GTAW), SMAW (stick); five injected fault types.

12. Evaluation & Results

All metrics are computed on the synthetic dataset under 5-fold stratified cross-validation (labels known exactly). Headline results:

Metric	Target	Result
Anomaly detection F1 (5-fold CV)	≥ 0.85	0.95
Precision / Recall	—	0.92 / 0.98
ROC-AUC	≥ 0.90	0.999
SHAP driver alignment (overall)	high	82% (100% for 4 of 5 fault types)
Root-cause ranking (MRR / Top-3)	—	0.878 / 100%
Parameter advisor in-window compliance	100%	100% (72 setpoints)
Confidence calibration (high band)	—	98% accurate
Fabricated numbers in answers	0	0
Unit tests passing	all	88 / 88

The optional fine-tuned DistilBERT detector matches the RandomForest baseline on the same train/test split, validating the transformer fine-tuning path. The near-perfect ROC-AUC reflects the cleanly separable synthetic faults and would be lower on noisier real-world data.

13. User Interfaces

- **Streamlit chat UI** — a dark industrial-themed conversational interface with parameter cards, SHAP charts, confidence badges, knowledge cards and retrieved-source panels.
- **FastAPI REST backend** — programmatic access via POST /chat, POST /pipeline/raw, and GET /health.

14. How to Run

```
# 1. install dependencies (Python 3.11 venv)
.venv\Scripts\python.exe -m pip install -r requirements.txt
```

```
# 2. configure a backend (copy and edit the env file)
copy .env.example .env          # set SYNTHESIZER_BACKEND and an API key

# 3. generate the synthetic dataset
.venv\Scripts\python.exe src\data\generate_synthetic.py

# 4. run the chatbot UI          -> http://localhost:8501
.venv\Scripts\python.exe -m streamlit run app\streamlit_app.py

# 5. (optional) run the REST API -> http://localhost:8000
.venv\Scripts\python.exe -m uvicorn src.api.main:app --reload

# 6. run the test suite          -> 88 passing
.venv\Scripts\python.exe -m pytest tests\ -q
```

15. Limitations & Future Work

Current limitations

- Validated on synthetic (not real factory) data; near-perfect AUC is optimistic for production noise.
- The LLM is narration-only (not fine-tuned for domain reasoning) — deliberate, to prevent hallucinated numbers.
- Single-fault-per-window detection; no visual/image weld-defect inspection.
- No physical hardware / PLC / SCADA integration or real-time closed-loop control.

Planned future work

- Validate on real welding-cell data and broaden materials/processes (titanium, FCAW, SAW).
- Vision-language model for weld-defect images; multi-label fault detection.
- Live MES / sensor-stream integration with a human-in-the-loop confirmation UI.
- Robotics / digital-twin integration (NVIDIA Isaac Sim) for sim-to-real welding-cell automation.

16. Conclusion

This project delivers a working MVP of an explainable, multimodal welding assistant that meets its core thesis goal: **transparent, traceable, grounded, hallucination-resistant** guidance. By strictly separating computation (welding physics and machine learning) from communication (the LLM), and by grounding every knowledge answer in cited retrieval, the system provides shop-floor advice that operators can trust — each recommendation backed by an explanation, a confidence score, or a source citation. The architecture is modular and data-agnostic, providing a solid foundation for the future work outlined above.